SPÉCIFICATION ICES v0.1

Le MissionBundle, la canonicalisation et le sceau Ed25519

Standard ouvert · Juin 2026 · Artefacts : `mission-bundle.schema.json`, `ices_verify.py`

ICES (IntrepidCore Engineering Standard) définit un **format de preuve d'intégrité de calcul d'ingénierie**. Un livrable ICES est un `MissionBundle` scellé par signature **asymétrique Ed25519**, qu'un tiers (bailleur, auditeur) vérifie **hors-ligne, sans secret partagé**. ICES est un **standard ouvert et gratuit** ; sa génération et sa vérification ne donnent lieu à aucune facturation.

1. Le MissionBundle

Objet JSON typé (schéma normatif : `mission-bundle.schema.json`, JSON Schema 2020-12), en cinq sections : `header`, `inputs`, `scenario` (optionnel), `outputs`, `gates`, plus le sceau `ices_envelope`. Un bundle *sans* `scenario` ni `ices_envelope` est l'état legacy déterministe : **rétrocompatibilité par construction**.

`header`, `gates` et `ices_envelope` sont **identiques quel que soit le produit émetteur** (LCPI, Atlas, IgnisCore). `inputs` et `outputs` portent une partie universelle (config, FallbackRegistry) et une partie **propre au domaine**, validée par des règles conditionnelles (`if header.domain == …then …`) dans le schéma normatif. C'est ce qui rend ICES nativement extensible à Atlas et IgnisCore sans dénaturer sa structure hydraulique d'origine.

Section	Contenu (résumé)
`header`	schema_version, project_key, **domain** (aep/hydro/atlas/igniscore/…), **engine_version** (identifiant neutre du moteur émetteur), date
`inputs`	**universel** : config canonisée, FallbackRegistry · **spécifique au domaine** : réseau (aep/hydro), geo_refs (IDs réalisations Atlas), geostatistics (atlas), biomass_mix (igniscore)
`scenario`	ensemble_seed, n_realizations, axes d'incertitude (sol, demande, IWS, coupures) — non applicable à `atlas` (déjà probabiliste via SGS)
`outputs`	**spécifique au domaine** : distribution/coûts (aep/hydro), realizations (atlas), biocore (igniscore) · robustness (P(service), équité) quand l'Ensemble Engine s'applique
`gates`	les contrôles EPS-001 (PASS/WARN/FAIL) — la preuve inclut aussi les échecs
`ices_envelope`	alg=Ed25519, digest SHA3-256, sig, **pubkey** (clé publique vérifiable)

2. Canonicalisation déterministe

Règle d'or : même entrée logique → mêmes octets, sur toute machine. Sinon le hash dérive et la preuve s'effondre.

— clés d'objet **triées** ; séparateurs compacts ; UTF-8 ;

— flottants **arrondis** à une précision fixée par champ ;

— chemins machine → **empreintes** (`sha`), jamais les chemins absolus ;

— géométrie **snappée** au préalable (logique de `canonize.py` généralisée).

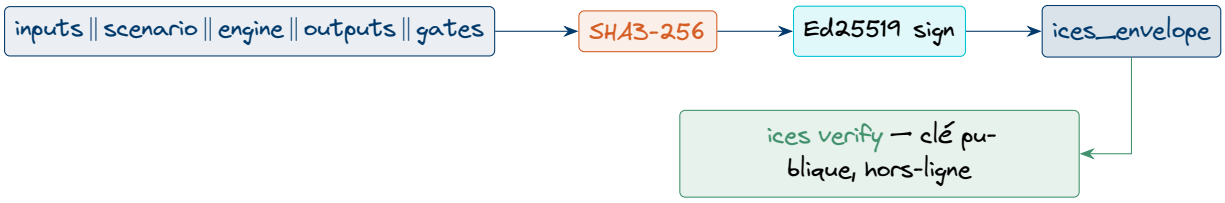
3. Le sceau

Au point de sortie unique du moteur (`finalize()`), quel que soit le produit :

$$\texttt{digest} = \text{SHA3-256}\big(\,\text{canon(inputs)} \,\|\, \text{canon(scenario)} \,\|\, \text{engine_version} \,\|\, \text{canon(outputs)} \,\|\, \text{canon(gates)}\,\big)$$

$$\texttt{sig} = \text{Ed25519.sign}(\texttt{digest},\ sk_{\text{émetteur}})$$

La clé privée sk reste chez l'émetteur ; la clé publique pk est diffusée. **Ed25519** (RFC 8032) est choisi pour son **déterminisme** (sceau reproductible) et son indépendance à tout aléa de signature.



4. Vérification (`ices verify`)

Le vérificateur de référence (`ices_verify.py` ; cible production : crate Rust `ices-core`, binaire offline) exécute trois étapes : (1) recanonicalise les sections signées ; (2) recalcule le condensat SHA3-256 ; (3) vérifie la signature Ed25519 avec la clé publique. Toute modification d'un extrant après signature fait diverger le condensat → **INVALIDE**.

Banc exécuté : un `MissionBundle` signé est déclaré `VALIDE` ; le même bundle dont on baisse le CAPEX *après* signature est déclaré `INVALIDE` (intégrité : ÉCHEC). La fraude post-calcul est détectée par un tiers, sans secret. *C'est la première des quatre preuves de structuration (jalon G1).*

5. Extension multi-domaine (Atlas, IgnisCore)

Le schéma v2 est délibérément **généralisé** : seuls `header`, `gates` et `ices_envelope` sont universels ; `inputs`/`outputs` varient par domaine via des règles conditionnelles (`allOf`/`if-then` sur `header.domain`). Un bundle `atlas` exige `inputs.geostatistics` et `outputs.realizations` ; un bundle `igniscore` exige `inputs.biomass_mix` et `outputs.biocore` ; un bundle `aep/hydro` exige le réseau et la distribution.

Un `MissionBundle` de domaine `atlas` (run géostatistique : variogramme, méthode SGS, IDs de sondage en entrée ; réalisations P10–P90 en sortie) a été signé, validé contre le schéma et vérifié `VALIDE`, au même titre qu'un bundle `aep`. Contre-épreuve : maquiller ce même bundle en `domain: aep` le fait **rejeter** par le schéma (champs hydrauliques requis absents) — la séparation enveloppe/charge utile fonctionne, elle n'est pas qu'une déclaration d'intention.

Cette généralisation porte le **schéma et le vérificateur**. Les codebases Atlas et IgnisCore n'exposent pas encore de point `finalize()` équivalent à celui de LCPI : le câblage réel (émission d'un bundle scellé à la fin d'un run de krigeage ou d'un dimensionnement BioCore) reste un chantier d'intégration à part entière, non encore réalisé. Ne pas surpromettre : le standard est prêt à les recevoir, il ne les couvre pas encore en production.

6. Statut & gouvernance

ICES v0.1 est publié en **standard ouvert**. La spécification, le schéma JSON et le vérificateur de référence sont libres d'accès. Gestion des clés : HSM/KMS pour la racine, clés FDE par mission. ICES ne certifie pas la responsabilité de l'ingénieur, mais l'**intégrité** du calcul (intrants, moteur, extrants).

ICES v0.1 · IntrepidCore · Artefacts normatifs : `mission-bundle.schema.json` · `ices_verify.py`